

4697566D My Rental Price SageMaker Pipeline V2

Singapore non-landed rental price estimation using XGBoost

The pipeline covers data cleaning, processing, training, evaluation and conditional registration flow, but incorporates the successful MLflow deployment pattern: a SageMaker `Model` is created with the trained artifact plus custom `inference.py`, and the model is registered. This lets the registered model deploy with the custom inference specification.

```
In [1]: # After running, restart the kernel before continuing if packages were upgraded.
%pip install --upgrade "sagemaker>=2,<3" boto3 botocore mlflow sagemaker-mlflow
```

```
Collecting sagemaker<3,>=2
  Using cached sagemaker-2.257.6-py3-none-any.whl.metadata (19 kB)
Requirement already satisfied: boto3 in /opt/conda/lib/python3.12/site-packages (1.43.46)
Collecting boto3
  Using cached boto3-1.43.80-py3-none-any.whl.metadata (6.6 kB)
Requirement already satisfied: botocore in /opt/conda/lib/python3.12/site-packages (1.43.46)
Collecting botocore
  Using cached botocore-1.43.80-py3-none-any.whl.metadata (5.6 kB)
Requirement already satisfied: mlflow in /opt/conda/lib/python3.12/site-packages (3.13.0)
Collecting mlflow
  Using cached mlflow-3.15.2-py3-none-any.whl.metadata (49 kB)
Requirement already satisfied: sagemaker-mlflow in /opt/conda/lib/python3.12/site-packages (0.5.0)
Collecting attrs<26,>=24 (from sagemaker<3,>=2)
  Using cached attrs-25.4.0-py3-none-any.whl.metadata (10 kB)
Requirement already satisfied: cloudpickle>=2.2.1 in /opt/conda/lib/python3.12/site-packages (from sagemaker<3,>=2) (3.1.2)
Requirement already satisfied: docker in /opt/conda/lib/python3.12/site-packages (from sagemaker<3,>=2) (7.2.0)
Requirement already satisfied: fastapi in /opt/conda/lib/python3.12/site-packages (from sagemaker<3,>=2) (0.139.2)
Requirement already satisfied: google-pasta in /opt/conda/lib/python3.12/site-packages (from sagemaker<3,>=2) (0.2.0)
Requirement already satisfied: graphene<4,>=3 in /opt/conda/lib/python3.12/site-packages (from sagemaker<3,>=2) (3.4.3)
Requirement already satisfied: importlib-metadata<7.0,>=1.4.0 in /opt/conda/lib/python3.12/site-packages (from sagemaker<3,>=2) (6.10.0)
Requirement already satisfied: jsonschema in /opt/conda/lib/python3.12/site-packages (from sagemaker<3,>=2) (4.23.0)
Requirement already satisfied: numpy<3.0,>=1.26.4 in /opt/conda/lib/python3.12/site-packages (from sagemaker<3,>=2) (1.26.4)
Requirement already satisfied: omegaconf<3,>=2.2 in /opt/conda/lib/python3.12/site-packages (from sagemaker<3,>=2) (2.3.0)
Collecting packaging<25,>=23.0 (from sagemaker<3,>=2)
  Using cached packaging-24.2-py3-none-any.whl.metadata (3.2 kB)
Requirement already satisfied: pandas>=2.3.0 in /opt/conda/lib/python3.12/site-packages (from sagemaker<3,>=2) (2.3.3)
Collecting pathos (from sagemaker<3,>=2)
  Using cached pathos-0.3.5-py3-none-any.whl.metadata (11 kB)
Requirement already satisfied: platformdirs in /opt/conda/lib/python3.12/site-packages (from sagemaker<3,>=2) (4.11.0)
Requirement already satisfied: protobuf<7.0,>=3.12 in /opt/conda/lib/python3.12/site-packages (from sagemaker<3,>=2) (6.31.1)
Requirement already satisfied: psutil in /opt/conda/lib/python3.12/site-packages (from sagemaker<3,>=2) (5.9.8)
Requirement already satisfied: pytz in /opt/conda/lib/python3.12/site-packages (from sagemaker<3,>=2) (2024.2)
Requirement already satisfied: pyyaml>=6.0.1 in /opt/conda/lib/python3.12/site-packages (from sagemaker<3,>=2) (6.0.3)
Requirement already satisfied: requests in /opt/conda/lib/python3.12/site-packages (from sagemaker<3,>=2) (2.34.2)
```

Collecting sagemaker-core<2.0.0,>=1.0.71 (from sagemaker<3,>=2)
Using cached sagemaker_core-1.0.78-py3-none-any.whl.metadata (4.9 kB)
Requirement already satisfied: schema in /opt/conda/lib/python3.12/site-packages (from sagemaker<3,>=2) (0.7.8)
Requirement already satisfied: smdebug-rulesconfig==1.0.1 in /opt/conda/lib/python3.12/site-packages (from sagemaker<3,>=2) (1.0.1)
Requirement already satisfied: tblib<4,>=1.7.0 in /opt/conda/lib/python3.12/site-packages (from sagemaker<3,>=2) (3.2.2)
Requirement already satisfied: tqdm in /opt/conda/lib/python3.12/site-packages (from sagemaker<3,>=2) (4.69.0)
Requirement already satisfied: urllib3<3.0.0,>=1.26.8 in /opt/conda/lib/python3.12/site-packages (from sagemaker<3,>=2) (1.26.20)
Requirement already satisfied: uvicorn in /opt/conda/lib/python3.12/site-packages (from sagemaker<3,>=2) (0.51.0)
Requirement already satisfied: jmespath<2.0.0,>=0.7.1 in /opt/conda/lib/python3.12/site-packages (from boto3) (1.1.0)
Requirement already satisfied: s3transfer<0.20.0,>=0.19.0 in /opt/conda/lib/python3.12/site-packages (from boto3) (0.19.1)
Requirement already satisfied: python-dateutil<3.0.0,>=2.1 in /opt/conda/lib/python3.12/site-packages (from botocore) (2.9.0.post0)
Requirement already satisfied: graphql-core<3.3,>=3.1 in /opt/conda/lib/python3.12/site-packages (from graphene<4,>=3->sagemaker<3,>=2) (3.2.11)
Requirement already satisfied: graphql-relay<3.3,>=3.1 in /opt/conda/lib/python3.12/site-packages (from graphene<4,>=3->sagemaker<3,>=2) (3.2.0)
Requirement already satisfied: typing-extensions<5,>=4.7.1 in /opt/conda/lib/python3.12/site-packages (from graphene<4,>=3->sagemaker<3,>=2) (4.16.0)
Requirement already satisfied: zipp>=0.5 in /opt/conda/lib/python3.12/site-packages (from importlib-metadata<7.0,>=1.4.0->sagemaker<3,>=2) (4.1.0)
Requirement already satisfied: antlr4-python3-runtime==4.9.* in /opt/conda/lib/python3.12/site-packages (from omegaconf<3,>=2.2->sagemaker<3,>=2) (4.9.3)
Requirement already satisfied: six>=1.5 in /opt/conda/lib/python3.12/site-packages (from python-dateutil<3.0.0,>=2.1->botocore) (1.17.0)
Requirement already satisfied: pydantic<3.0.0,>=2.0.0 in /opt/conda/lib/python3.12/site-packages (from sagemaker-core<2.0.0,>=1.0.71->sagemaker<3,>=2) (2.13.4)
Requirement already satisfied: rich<15.0.0,>=13.0.0 in /opt/conda/lib/python3.12/site-packages (from sagemaker-core<2.0.0,>=1.0.71->sagemaker<3,>=2) (14.3.4)
Requirement already satisfied: mock<5.0,>=4.0 in /opt/conda/lib/python3.12/site-packages (from sagemaker-core<2.0.0,>=1.0.71->sagemaker<3,>=2) (4.0.3)
Requirement already satisfied: jsonschema-specifications>=2023.03.6 in /opt/conda/lib/python3.12/site-packages (from jsonschema->sagemaker<3,>=2) (2025.9.1)
Requirement already satisfied: referencing>=0.28.4 in /opt/conda/lib/python3.12/site-packages (from jsonschema->sagemaker<3,>=2) (0.37.0)
Requirement already satisfied: rpds-py>=0.7.1 in /opt/conda/lib/python3.12/site-packages (from jsonschema->sagemaker<3,>=2) (0.27.1)
Requirement already satisfied: annotated-types>=0.6.0 in /opt/conda/lib/python3.12/site-packages (from pydantic<3.0.0,>=2.0.0->sagemaker-core<2.0.0,>=1.0.71->sagemaker<3,>=2) (0.7.0)
Requirement already satisfied: pydantic-core==2.46.4 in /opt/conda/lib/python3.12/site-packages (from pydantic<3.0.0,>=2.0.0->sagemaker-core<2.0.0,>=1.0.71->sagemaker<3,>=2) (2.46.4)
Requirement already satisfied: typing-inspection>=0.4.2 in /opt/conda/lib/python3.12/site-packages (from pydantic<3.0.0,>=2.0.0->sagemaker-core<2.0.0,>=1.0.71->sagemaker<3,>=2) (0.4.2)
Requirement already satisfied: markdown-it-py>=2.2.0 in /opt/conda/lib/python3.12/site-packages (from rich<15.0.0,>=13.0.0->sagemaker-core<2.0.0,>=1.0.71->sagemaker<3,>=2) (4.2.0)
Requirement already satisfied: pygments<3.0.0,>=2.13.0 in /opt/conda/lib/python3.12/site-packages (from rich<15.0.0,>=13.0.0->sagemaker-core<2.0.0,>=1.0.71->sagemaker<3,>=2) (2.20.0)

Collecting mlflow-skinny==3.15.2 (from mlflow)
Using cached mlflow_skinny-3.15.2-py3-none-any.whl.metadata (50 kB)
Collecting mlflow-tracing==3.15.2 (from mlflow)
Using cached mlflow_tracing-3.15.2-py3-none-any.whl.metadata (19 kB)
Requirement already satisfied: Flask-CORS<7 in /opt/conda/lib/python3.12/site-packages (from mlflow) (6.0.5)
Requirement already satisfied: Flask<4 in /opt/conda/lib/python3.12/site-packages (from mlflow) (3.1.3)
Requirement already satisfied: aiohttp<4,>=3.7.0 in /opt/conda/lib/python3.12/site-packages (from mlflow) (3.14.2)
Requirement already satisfied: alembic!=1.10.0,<2 in /opt/conda/lib/python3.12/site-packages (from mlflow) (1.18.5)
Requirement already satisfied: cryptography<50,>=43.0.0 in /opt/conda/lib/python3.12/site-packages (from mlflow) (46.0.7)
Requirement already satisfied: gunicorn<27 in /opt/conda/lib/python3.12/site-packages (from mlflow) (23.0.0)
Requirement already satisfied: huey<4,>=2.5.4 in /opt/conda/lib/python3.12/site-packages (from mlflow) (2.6.0)
Requirement already satisfied: matplotlib<4 in /opt/conda/lib/python3.12/site-packages (from mlflow) (3.10.9)
Requirement already satisfied: pyarrow<26,>=4.0.0 in /opt/conda/lib/python3.12/site-packages (from mlflow) (21.0.0)
Requirement already satisfied: scikit-learn<2 in /opt/conda/lib/python3.12/site-packages (from mlflow) (1.7.2)
Requirement already satisfied: scipy<2 in /opt/conda/lib/python3.12/site-packages (from mlflow) (1.16.3)
Requirement already satisfied: skops<1 in /opt/conda/lib/python3.12/site-packages (from mlflow) (0.14.0)
Requirement already satisfied: sqlalchemy<3,>=1.4.0 in /opt/conda/lib/python3.12/site-packages (from mlflow) (2.0.51)
Requirement already satisfied: cachetools<8,>=5.0.0 in /opt/conda/lib/python3.12/site-packages (from mlflow-skinny==3.15.2->mlflow) (6.2.6)
Requirement already satisfied: click<9,>=7.0 in /opt/conda/lib/python3.12/site-packages (from mlflow-skinny==3.15.2->mlflow) (8.2.1)
Requirement already satisfied: databricks-sdk<1,>=0.20.0 in /opt/conda/lib/python3.12/site-packages (from mlflow-skinny==3.15.2->mlflow) (0.122.0)

Requirement already satisfied: gitpython<4,>=3.1.9 in /opt/conda/lib/python3.12/site-packages (from mlflow-skinny==3.15.2->mlflow) (3.1.53)

Requirement already satisfied: opentelemetry-api<3,>=1.9.0 in /opt/conda/lib/python3.12/site-packages (from mlflow-skinny==3.15.2->mlflow) (1.43.0)

Requirement already satisfied: opentelemetry-proto<3,>=1.9.0 in /opt/conda/lib/python3.12/site-packages (from mlflow-skinny==3.15.2->mlflow) (1.44.0)

Requirement already satisfied: opentelemetry-sdk<3,>=1.9.0 in /opt/conda/lib/python3.12/site-packages (from mlflow-skinny==3.15.2->mlflow) (1.43.0)

Requirement already satisfied: python-dotenv<2,>=0.19.0 in /opt/conda/lib/python3.12/site-packages (from mlflow-skinny==3.15.2->mlflow) (1.2.2)

Requirement already satisfied: sqlparse<1,>=0.4.0 in /opt/conda/lib/python3.12/site-packages (from mlflow-skinny==3.15.2->mlflow) (0.5.0)

Requirement already satisfied: starlette<2 in /opt/conda/lib/python3.12/site-packages (from mlflow-skinny==3.15.2->mlflow) (0.52.1)

Requirement already satisfied: aiohappyeyeballs>=2.5.0 in /opt/conda/lib/python3.12/site-packages (from aiohttp<4,>=3.7.0->mlflow) (2.7.1)

Requirement already satisfied: aiosignal>=1.4.0 in /opt/conda/lib/python3.12/site-packages (from aiohttp<4,>=3.7.0->mlflow) (1.4.0)

Requirement already satisfied: frozenlist>=1.1.1 in /opt/conda/lib/python3.12/site-packages (from aiohttp<4,>=3.7.0->mlflow) (1.8.0)

Requirement already satisfied: multidict<7.0,>=4.5 in /opt/conda/lib/python3.12/site-packages (from aiohttp<4,>=3.7.0->mlflow) (6.7.1)

Requirement already satisfied: propcache>=0.2.0 in /opt/conda/lib/python3.12/site-packages (from aiohttp<4,>=3.7.0->mlflow) (0.5.2)

Requirement already satisfied: yarl<2.0,>=1.17.0 in /opt/conda/lib/python3.12/site-packages (from aiohttp<4,>=3.7.0->mlflow) (1.24.5)

Requirement already satisfied: Mako in /opt/conda/lib/python3.12/site-packages (from alembic!=1.10.0,<2->mlflow) (1.3.12)

Requirement already satisfied: cffi>=1.14 in /opt/conda/lib/python3.12/site-packages (from cryptography<50,>=43.0.0->mlflow) (1.17.1)

Requirement already satisfied: google-auth~=2.0 in /opt/conda/lib/python3.12/site-packages (from databricks-sdk<1,>=0.20.0->mlflow-skinny==3.15.2->mlflow) (2.56.0)

Requirement already satisfied: annotated-doc>=0.0.2 in /opt/conda/lib/python3.12/site-packages (from fastapi->sagemaker<3,>=2) (0.0.4)

Requirement already satisfied: blinker>=1.9.0 in /opt/conda/lib/python3.12/site-packages (from Flask<4->mlflow) (1.9.0)

Requirement already satisfied: itsdangerous>=2.2.0 in /opt/conda/lib/python3.12/site-packages (from Flask<4->mlflow) (2.2.0)

Requirement already satisfied: jinja2>=3.1.2 in /opt/conda/lib/python3.12/site-packages (from Flask<4->mlflow) (3.1.6)

Requirement already satisfied: markupsafe>=2.1.1 in /opt/conda/lib/python3.12/site-packages (from Flask<4->mlflow) (3.0.3)

Requirement already satisfied: werkzeug>=3.1.0 in /opt/conda/lib/python3.12/site-packages (from Flask<4->mlflow) (3.1.8)

Requirement already satisfied: gitdb<5,>=4.0.1 in /opt/conda/lib/python3.12/site-packages (from gitpython<4,>=3.1.9->mlflow-skinny==3.15.2->mlflow) (4.0.12)

Requirement already satisfied: smmap<6,>=3.0.1 in /opt/conda/lib/python3.12/site-packages (from gitdb<5,>=4.0.1->gitpython<4,>=3.1.9->mlflow-skinny==3.15.2->mlflow) (5.0.3)

Requirement already satisfied: pyasn1-modules>=0.2.1 in /opt/conda/lib/python3.12/site-packages (from google-auth~=2.0->databricks-sdk<1,>=0.20.0->mlflow-skinny==3.15.2->mlflow) (0.4.2)

Requirement already satisfied: contourpy>=1.0.1 in /opt/conda/lib/python3.12/site-packages (from matplotlib<4->mlflow) (1.3.3)

Requirement already satisfied: cycycler>=0.10 in /opt/conda/lib/python3.12/site-packages (from matplotlib<4->mlflow) (0.12.1)

Requirement already satisfied: fonttools>=4.22.0 in /opt/conda/lib/python3.12/site-packages (from matplotlib<4->mlflow) (4.63.0)

Requirement already satisfied: kiwisolver>=1.3.1 in /opt/conda/lib/python3.12/site-packages (from matplotlib<4->mlflow) (1.5.0)

Requirement already satisfied: pillow>=8 in /opt/conda/lib/python3.12/site-packages (from matplotlib<4->mlflow) (11.3.0)

Requirement already satisfied: pyparsing>=3 in /opt/conda/lib/python3.12/site-packages (from matplotlib<4->mlflow) (3.3.2)

Requirement already satisfied: opentelemetry-semantic-conventions==0.64b0 in /opt/conda/lib/python3.12/site-packages (from opentelemetry-sdk<3,>=1.9.0->mlflow-skinny==3.15.2->mlflow) (0.64b0)

Requirement already satisfied: tzdata>=2022.7 in /opt/conda/lib/python3.12/site-packages (from pandas>=2.3.0->sagemaker<3,>=2) (2026.3)

Requirement already satisfied: charset_normalizer<4,>=2 in /opt/conda/lib/python3.12/site-packages (from requests->sagemaker<3,>=2) (3.4.9)

Requirement already satisfied: idna<4,>=2.5 in /opt/conda/lib/python3.12/site-packages (from requests->sagemaker<3,>=2) (3.18)

Requirement already satisfied: certifi>=2023.5.7 in /opt/conda/lib/python3.12/site-packages (from requests->sagemaker<3,>=2) (2026.6.17)

Requirement already satisfied: joblib>=1.2.0 in /opt/conda/lib/python3.12/site-packages (from scikit-learn<2->mlflow) (1.5.3)

Requirement already satisfied: threadpoolctl>=3.1.0 in /opt/conda/lib/python3.12/site-packages (from scikit-learn<2->mlflow) (3.6.0)

Collecting prettytable>=3.9 (from skops<1->mlflow)

Using cached prettytable-3.18.0-py3-none-any.whl.metadata (37 kB)

Requirement already satisfied: greenlet>=1 in /opt/conda/lib/python3.12/site-packages (from sqlalchemy<3,>=1.4.0->mlflow) (3.5.3)

Requirement already satisfied: anyio<5,>=3.6.2 in /opt/conda/lib/python3.12/site-packages (from starlette<2->mlflow-skinny==3.15.2->mlflow) (4.14.2)

Requirement already satisfied: h11>=0.8 in /opt/conda/lib/python3.12/site-packages (from uvicorn->sagemaker<3,>=2) (0.16.0)

Requirement already satisfied: pycparser in /opt/conda/lib/python3.12/site-packages (from cffi>=1.14-> cryptography<50,>=43.0.0->mlflow) (3.0)

Requirement already satisfied: mdurl~=0.1 in /opt/conda/lib/python3.12/site-packages (from markdown-it-py>=2.2.0->rich<15.0.0,>=13.0.0->sagemaker-core<2.0.0,>=1.0.71->sagemaker<3,>=2) (0.1.2)

Requirement already satisfied: wcwidth>=0.3.5 in /opt/conda/lib/python3.12/site-packages (from prettytable>=3.9->skops<1->mlflow) (0.8.2)

Requirement already satisfied: pyasn1<0.7.0,>=0.6.1 in /opt/conda/lib/python3.12/site-packages (from pyasn1-modules>=0.2.1->google-auth~2.0->databricks-sdk<1,>=0.20.0->mlflow-skinny==3.15.2->mlflow) (0.6.4)

Collecting ppft>=1.7.8 (from pathos->sagemaker<3,>=2)

Using cached ppft-1.7.8-py3-none-any.whl.metadata (12 kB)

Requirement already satisfied: dill>=0.4.1 in /opt/conda/lib/python3.12/site-packages (from pathos->sagemaker<3,>=2) (0.4.1)

Collecting pox>=0.3.7 (from pathos->sagemaker<3,>=2)

Using cached pox-0.3.7-py3-none-any.whl.metadata (8.0 kB)

Requirement already satisfied: multiprocessing>=0.70.19 in /opt/conda/lib/python3.12/site-packages (from pathos->sagemaker<3,>=2) (0.70.19)

Using cached sagemaker-2.257.6-py3-none-any.whl (1.7 MB)

Using cached boto3-1.43.80-py3-none-any.whl (140 kB)

Using cached botocore-1.43.80-py3-none-any.whl (15.7 MB)

Using cached attrs-25.4.0-py3-none-any.whl (67 kB)

Using cached packaging-24.2-py3-none-any.whl (65 kB)

Using cached sagemaker_core-1.0.78-py3-none-any.whl (444 kB)

Using cached mlflow-3.15.2-py3-none-any.whl (11.2 MB)

Using cached mlflow_skinny-3.15.2-py3-none-any.whl (3.6 MB)

Using cached mlflow_tracing-3.15.2-py3-none-any.whl (1.8 MB)

Using cached prettytable-3.18.0-py3-none-any.whl (37 kB)

Using cached pathos-0.3.5-py3-none-any.whl (82 kB)

Using cached pox-0.3.7-py3-none-any.whl (29 kB)

Using cached ppft-1.7.8-py3-none-any.whl (56 kB)

Installing collected packages: prettytable, ppft, pox, packaging, attrs, pathos, botocore, boto3, sagemaker-core, mlflow-tracing, mlflow-skinny, sagemaker, mlflow

Attempting uninstall: packaging

Found existing installation: packaging 25.0

Uninstalling packaging-25.0:

Successfully uninstalled packaging-25.0

[illegible]


```
print("SageMaker SDK:", sagemaker.__version__)
```

sagemaker.config INFO - Not applying SDK defaults from location: /etc/xdg/sagemaker/config.yaml
sagemaker.config INFO - Not applying SDK defaults from location: /home/sagemaker-user/.config/sagemaker/config.yaml

/opt/conda/lib/python3.12/site-packages/sagemaker/__init__.py:86: SageMakerV2DeprecationWarning: You are using the SageMaker Python SDK v2, which is on path of deprecation. v3 is the actively developed major version.

See <https://github.com/aws/sagemaker-python-sdk/blob/master/migration.md> for the migration guide. Set SAGEMAKER_SUPPRESS_V2_WARNING=1 to silence this warning.

```
warn_v2_deprecation()
```

You are using the SageMaker Python SDK v2, which is on path of deprecation. v3 is the actively developed major version.

See <https://github.com/aws/sagemaker-python-sdk/blob/master/migration.md> for the migration guide. Set SAGEMAKER_SUPPRESS_V2_WARNING=1 to silence this warning.

SageMaker SDK: 2.257.6

1. Configuration

```
In [3]: sess = boto3.Session()
role = get_execution_role()
sagemaker_session = sagemaker.Session(boto_session=sess)
pipeline_session = PipelineSession()
region = sess.region_name
bucket = sagemaker_session.default_bucket()

# Change these for the current student/profile.
TEAM_ID = "Team08"
STUDENT_ID = "s801"

COURSE = "ITI113"
SEMESTER = "26S1"

PROJECT_NAME = "4697566D-Rental-Pipeline-V2"
PIPELINE_NAME = PROJECT_NAME
MODEL_PACKAGE_GROUP = "RentalPriceModelGroupV2"
BASE_PREFIX = "sgp-rental-v2"

TEAM_ID = "Team08"
STUDENT_ID = "s801"

COURSE = "ITI113"
SEMESTER = "26S1"

PROCESSING_INSTANCE = "ml.m5.large"
TRAINING_INSTANCE = "ml.m5.large"
INFERENCE_INSTANCE = "ml.m5.large"

print("Region:", region)
print("Bucket:", bucket)
print("Role:", role)
print("Model group:", MODEL_PACKAGE_GROUP)

# -----
# MLflow App UI Link helpers
# -----
# MLflow may print generic links such as:
# https://mlflow.sagemaker.ap-southeast-1.app.aws/#/...
# Those links are not presigned and may show a permission/session error.
# Use these helpers to generate a fresh presigned SageMaker MLflow App URL
# and append the experiment/run fragment.
MLFLOW_EXPERIMENT_NAME = f"{COURSE}/{TEAM_ID}/Experiment"

def create_mlflow_app_presigned_url(fragment: str = "") -> str:
    sm_for_mlflow = boto3.client("sagemaker", region_name=region)
    response = sm_for_mlflow.create_presigned_mlflow_app_url(
        Arn=MLFLOW_APP_ARN
    )

    base_url = response.get("AuthorizedUrl") or response.get("Url")
```



```

    if not base_url:
        raise RuntimeError(
            "create_presigned_mlflow_app_url did not return AuthorizedUrl or Url. "
            f"Response: {response}"
        )

    # Remove any existing fragment before appending our own MLflow UI route.
    base_url = base_url.split("#", 1)[0]

    if fragment:
        return base_url + "#" + fragment.lstrip("#")

    return base_url

def print_mlflow_presigned_links(experiment_id=None, run_id=None):
    if experiment_id is not None:
        experiment_url = create_mlflow_app_presigned_url(
            f"/experiments/{experiment_id}"
        )
        print("Presigned MLflow experiment URL:")
        print(experiment_url)

    if experiment_id is not None and run_id is not None:
        run_url = create_mlflow_app_presigned_url(
            f"/experiments/{experiment_id}/runs/{run_id}"
        )
        print("\nPresigned MLflow run URL:")
        print(run_url)

```

Region: ap-southeast-1

Bucket: sagemaker-ap-southeast-1-044528205969

Role: arn:aws:iam::044528205969:role/SageMakerExecutionRole-IT1113-Team08

Model group: RentalPriceModelGroupV2

```

In [4]: import boto3
import time
import json
from botocore.exceptions import ClientError

MLFLOW_APP_NAME = "iti113-26s1-team08-mlflow-app"

REGION = "ap-southeast-1"
MLFLOW_APP_ARN = "arn:aws:sagemaker:ap-southeast-1:044528205969:mlflow-app/app-WLHEEZL7YYAH"

sm = boto3.client("sagemaker", region_name=REGION)

while True:
    try:
        response = sm.describe_mlflow_app(
            Arn=MLFLOW_APP_ARN
        )

        print("=" * 80)
        print("Name:", response.get("Name"))
        print("Status:", response.get("Status"))
        print("Arn:", response.get("Arn"))
        print("LastModifiedTime:", response.get("LastModifiedTime"))

        if response.get("Status") in ["Created", "Failed", "Deleting", "Deleted"]:
            break

    except ClientError as e:
        print("Describe failed.")
        print(e.response["Error"]["Code"])
        print(e.response["Error"]["Message"])
        break

    time.sleep(30)

```

```

=====
Name: iti113-26s1-team08-mlflow-app
Status: Created
Arn: arn:aws:sagemaker:ap-southeast-1:044528205969:mlflow-app/app-WLHEEZL7YYAH
LastModifiedTime: 2026-08-19 13:01:50.294000+00:00

```

```
In [5]: import boto3
import time
import json
from botocore.exceptions import ClientError

REGION = "ap-southeast-1"
ACCOUNT_ID = "044528205969"
MLFLOW_APP_ARN = "arn:aws:sagemaker:ap-southeast-1:044528205969:mlflow-app/app-WLHEEZL7YYAH"

TEAM_ID = "team08"
STUDENT_ID = "s801"

MLFLOW_APP_NAME = "iti113-26s1-team08-mlflow-app"
ARTIFACT_STORE_URI = "s3://nyp-26s1-iti113/iti113/team08/mlflow-app-artifacts/"
ROLE_ARN = f"arn:aws:iam::{ACCOUNT_ID}:role/SageMakerExecutionRole-ITI113-Team08"

sm = boto3.client("sagemaker", region_name=REGION)

while True:
    try:
        response = sm.describe_mlflow_app(
            Arn=MLFLOW_APP_ARN
        )

        print("=" * 80)
        print("Name:", response.get("Name"))
        print("Status:", response.get("Status"))
        print("Arn:", response.get("Arn"))
        print("LastModifiedTime:", response.get("LastModifiedTime"))

        if response.get("Status") in ["Created", "Failed", "Deleting", "Deleted"]:
            break

    except ClientError as e:
        print("Describe failed.")
        print(e.response["Error"]["Code"])
        print(e.response["Error"]["Message"])
        break

    time.sleep(30)

=====
Name: iti113-26s1-team08-mlflow-app
Status: Created
Arn: arn:aws:sagemaker:ap-southeast-1:044528205969:mlflow-app/app-WLHEEZL7YYAH
LastModifiedTime: 2026-08-19 13:01:50.294000+00:00
```

2. Prepare and upload the cleaned dataset##

```
In [6]: import numpy as np
import pandas as pd

#upload to Jupyter Server

rental = pd.read_csv("rent21_25_yr.csv", index_col=0)

rental.rename(columns={
    "Project Name": "project",
    "Street Name": "street",
    "Postal District": "district",
    "Property Type": "type",
    "No of Bedroom": "bedroom",
    "Monthly Rent ($)": "mth rent",
    "Floor Area (SQM)": "area_sm",
    "Floor Area (SQFT)": "area_sf",
    "Lease Commencement Date": "lease date"
}, inplace=True)

rental_cl = rental.dropna().copy()
rental_cl.drop(columns=["type", "area_sf", "errors"], errors="ignore", inplace=True)
rental_cl["lease date"] = pd.to_datetime(rental_cl["lease date"], format="%b-%y")
rental_cl["lease_year"] = rental_cl["lease date"].dt.year
```

```

rental_cl["lease_month"] = rental_cl["lease_date"].dt.month
rental_cl.drop(columns=["lease_date"], inplace=True)

rental_cl["area_sm"] = (
    rental_cl["area_sm"].astype(str).str.split("to", expand=True)[1].str.strip()
)

rental_cl = rental_cl[
    pd.to_numeric(rental_cl["district"], errors="coerce").notnull() &
    pd.to_numeric(rental_cl["bedroom"], errors="coerce").notnull() &
    pd.to_numeric(rental_cl["area_sm"], errors="coerce").notnull()
].copy()

rental_cl["district"] = rental_cl["district"].astype(int)
rental_cl["bedroom"] = rental_cl["bedroom"].astype(int)
rental_cl["area_sm"] = rental_cl["area_sm"].astype(int)
rental_cl["mth rent"] = rental_cl["mth rent"].astype(str).str.replace(",", "", regex=False)
rental_cl = rental_cl[pd.to_numeric(rental_cl["mth rent"], errors="coerce").notnull()].copy()
rental_cl["mth rent"] = (rental_cl["mth rent"].astype(float) / 1000).round(3)

rental_2125 = rental_cl.copy()
rental_2125["price_per_sqm"] = (rental_2125["mth rent"] * 1000 / rental_2125["area_sm"]).round(3)

rental_2125.to_csv("rental_2125.csv", index=False)
print("Rows:", len(rental_2125))
print("Columns:", list(rental_2125.columns))

input_data_uri = sagemaker_session.upload_data(
    path="rental_2125.csv",
    bucket=bucket,
    key_prefix=f"{BASE_PREFIX}/input"
)
print("Input S3:", input_data_uri)

```

Rows: 368897

Columns: ['project', 'street', 'district', 'bedroom', 'mth rent', 'area_sm', 'lease_year', 'lease_month', 'price_per_sqm']

Input S3: s3://sagemaker-ap-southeast-1-044528205969/sgp-rental-v2/input/rental_2125.csv

In [7]: *## 3. Pipeline parameters*

```

In [8]: from sagemaker.workflow.parameters import ParameterString, ParameterFloat

input_data = ParameterString(name="InputData", default_value=input_data_uri)
processing_instance_type = ParameterString(
    name="ProcessingInstanceType", default_value=PROCESSING_INSTANCE
)
training_instance_type = ParameterString(
    name="TrainingInstanceType", default_value=TRAINING_INSTANCE
)
model_approval_status = ParameterString(
    name="ModelApprovalStatus", default_value="Approved"
)
r2_threshold = ParameterFloat(name="R2Threshold", default_value=0.8)

```

4. Create the source directory

In [9]: `mkdir -p src`

5. Preprocessing script

```

In [10]: %%writefile src/preprocess.py
import pandas as pd
import numpy as np
import os
from sklearn.preprocessing import OneHotEncoder

if __name__ == "__main__":
    input_dir = "/opt/ml/processing/input"
    input_file = [f for f in os.listdir(input_dir) if f.endswith(".csv")][0]
    df = pd.read_csv(os.path.join(input_dir, input_file))

```

```

dist_dummy_cols = [c for c in df.columns if c.startswith("dist_")]
columns_to_drop = ["project", "street", "price_per_sqm"] + dist_dummy_cols
df_cleaned = df.drop(columns=columns_to_drop, errors="ignore")

target_col = "mth rent"
shuffled = df_cleaned.sample(frac=1, random_state=42).reset_index(drop=True)
n = len(shuffled)
train_end = int(0.70 * n)
val_end = int(0.85 * n)
train_df = shuffled.iloc[:train_end]
val_df = shuffled.iloc[train_end:val_end]
test_df = shuffled.iloc[val_end:]

encoder = OneHotEncoder(handle_unknown="ignore", sparse_output=False)
encoder.fit(train_df[["district"]])
district_cols = [f"dist_{int(c)}" for c in encoder.categories_[0]]

def build_output(split_df):
    district_encoded = pd.DataFrame(
        encoder.transform(split_df[["district"]]).astype(int),
        columns=district_cols,
        index=split_df.index,
    )
    numeric_cols = ["bedroom", "area_sm", "lease_year", "lease_month"]
    return pd.concat(
        [split_df[[target_col] + numeric_cols].reset_index(drop=True),
         district_encoded.reset_index(drop=True)], axis=1
    )

train_out = build_output(train_df)
val_out = build_output(val_df)
test_out = build_output(test_df)

for d in ["/opt/ml/processing/train", "/opt/ml/processing/validation", "/opt/ml/processing/test"]:
    os.makedirs(d, exist_ok=True)

train_out.to_csv("/opt/ml/processing/train/train.csv", header=False, index=False)
val_out.to_csv("/opt/ml/processing/validation/validation.csv", header=False, index=False)
test_out.to_csv("/opt/ml/processing/test/test.csv", header=False, index=False)

print(f"Preprocessing finished: train={len(train_out)}, validation={len(val_out)}, test={len(test_out)}")
print("Feature columns:", list(train_out.columns[1:]))

```

Overwriting src/preprocess.py

```

In [11]: from sagemaker.processing import ScriptProcessor, ProcessingInput, ProcessingOutput
from sagemaker.workflow.steps import ProcessingStep

sklearn_image = sagemaker.image_uris.retrieve(
    framework="sklearn", region=region, version="1.2-1"
)

processor = ScriptProcessor(
    image_uri=sklearn_image,
    command=["python3"],
    instance_type=processing_instance_type,
    instance_count=1,
    role=role,
    sagemaker_session=pipeline_session,
)

step_process = ProcessingStep(
    name="PreprocessData",
    processor=processor,
    inputs=[ProcessingInput(source=input_data, destination="/opt/ml/processing/input")],
    outputs=[
        ProcessingOutput(output_name="train", source="/opt/ml/processing/train"),
        ProcessingOutput(output_name="validation", source="/opt/ml/processing/validation"),
        ProcessingOutput(output_name="test", source="/opt/ml/processing/test"),
    ],
    code="src/preprocess.py",
)

```

```
/opt/conda/lib/python3.12/site-packages/sagemaker/processing.py:138: SageMakerV2DeprecationWarning: ScriptProcessor is part of the SageMaker Python SDK v2, which is on path of deprecation. In v3, use `DataProcessor` (`from sagemaker.mlops.processing import DataProcessor`).
See https://github.com/aws/sagemaker-python-sdk/blob/master/migration.md for the migration guide. Set SAGEMAKER_SUPPRESS_V2_WARNING=1 to silence this warning.
    warn_v2_deprecation(
ScriptProcessor is part of the SageMaker Python SDK v2, which is on path of deprecation. In v3, use `DataProcessor` (`from sagemaker.mlops.processing import DataProcessor`).
See https://github.com/aws/sagemaker-python-sdk/blob/master/migration.md for the migration guide. Set SAGEMAKER_SUPPRESS_V2_WARNING=1 to silence this warning.
```

6. XGBoost training script

```
In [12]: import mlflow
print("MLflow version:", mlflow.__version__)
```

MLflow version: 3.15.2

```
In [13]: %%writefile src/train.py
import argparse
import os
import sys
import subprocess

# The SageMaker XGBoost container does not necessarily include the
# SageMaker MLflow integration packages, so install them in the training
# container before importing MLflow.
subprocess.check_call(
    [sys.executable, "-m", "pip", "install", "-q", "sagemaker-mlflow"]
)

import pandas as pd
import xgboost as xgb
import numpy as np
from sklearn.metrics import mean_squared_error, r2_score, mean_absolute_error

import mlflow
import mlflow.xgboost

def configure_mlflow():
    """Connect this training job to the SageMaker MLflow App."""
    mlflow_app_arn = os.environ.get("MLFLOW_APP_ARN")
    experiment_name = os.environ.get(
        "MLFLOW_EXPERIMENT_NAME",
        "4697566D-Rental-Pipeline-V2-MLflow",
    )

    if not mlflow_app_arn:
        raise RuntimeError(
            "MLFLOW_APP_ARN is not set. The SageMaker training step "
            "must pass the MLflow App ARN in the environment."
        )

    # IMPORTANT:
    # This ARN is an MLflow App ARN:
    # arn:aws:sagemaker:<region>:<account>:mlflow-app/<id>
    # It is NOT an MLflow Tracking Server ARN.
    mlflow.set_tracking_uri(mlflow_app_arn)
    mlflow.set_experiment(experiment_name)

    return experiment_name

if __name__ == "__main__":
    parser = argparse.ArgumentParser()
    parser.add_argument("--eta", type=float, default=0.2)
    parser.add_argument("--max_depth", type=int, default=6)
    parser.add_argument("--num_round", type=int, default=400)
    parser.add_argument("--objective", type=str, default="reg:squarederror")
    parser.add_argument("--subsample", type=float, default=0.8)
    parser.add_argument("--eval_metric", type=str, default="rmse")
    parser.add_argument("--train", type=str, default=os.environ.get("SM_CHANNEL_TRAIN"))
    parser.add_argument("--validation", type=str, default=os.environ.get("SM_CHANNEL_VALIDATION"))
```

```

parser.add_argument("--model-dir", type=str, default=os.environ.get("SM_MODEL_DIR", "/opt/ml/model"))
args, _ = parser.parse_known_args()

train_df = pd.read_csv(
    os.path.join(args.train, "train.csv"), header=None
)
val_df = pd.read_csv(
    os.path.join(args.validation, "validation.csv"), header=None
)

x_train, y_train = train_df.iloc[:, 1:], train_df.iloc[:, 0]
x_val, y_val = val_df.iloc[:, 1:], val_df.iloc[:, 0]

model = xgb.XGBRegressor(
    n_estimators=args.num_round,
    learning_rate=args.eta,
    max_depth=args.max_depth,
    subsample=args.subsample,
    objective=args.objective,
    eval_metric=args.eval_metric,
    random_state=42,
    verbosity=1,
)

# -----
# MLflow tracking
# -----
experiment_name = configure_mlflow()

training_job_name = os.environ.get(
    "TRAINING_JOB_NAME",
    os.environ.get("SAGEMAKER_JOB_NAME", "rental-xgboost-training"),
)

with mlflow.start_run(run_name=training_job_name) as run:

    # Record the hyperparameters used by the XGBoost model.
    mlflow.log_params({
        "eta": args.eta,
        "max_depth": args.max_depth,
        "num_round": args.num_round,
        "objective": args.objective,
        "subsample": args.subsample,
        "eval_metric": args.eval_metric,
        "random_state": 42,
    })

    # Record useful dataset information.
    mlflow.log_params({
        "training_samples": len(train_df),
        "validation_samples": len(val_df),
        "features": x_train.shape[1],
    })

    mlflow.set_tags({
        "project": "4697566D My Rental Price",
        "pipeline": "4697566D-Rental-Pipeline-V2",
        "framework": "XGBoost",
        "sagemaker_training_job": training_job_name,
    })

    # Train the model.
    model.fit(x_train, y_train)

    # Validate the model.
    pred = model.predict(x_val)

    rmse = float(np.sqrt(mean_squared_error(y_val, pred)))
    mae = float(mean_absolute_error(y_val, pred))
    r2 = float(r2_score(y_val, pred))

    # Record evaluation metrics in MLflow.
    mlflow.log_metrics({
        "rmse": rmse,
        "mae": mae,

```

```

        "r2": r2,
    })

    # Save the same SageMaker model artifact used by the existing
    # RegisterModel/deployment steps.
    os.makedirs(args.model_dir, exist_ok=True)

    model.save_model(
        os.path.join(args.model_dir, "xgboost-model.ubj")
    )
    model.save_model(
        os.path.join(args.model_dir, "xgboost-model.json")
    )

    # Also record the trained model in the MLflow App.
    mlflow.xgboost.log_model(
        model,
        name="xgboost-model",
        registered_model_name="ITI113-Team08-SGP-Rental-Price-Model",
    )

    print("MLflow experiment:", experiment_name)
    print("MLflow run ID:", run.info.run_id)

    print(f"RMSE={rmse:.4f}")
    print(f"MAE={mae:.4f}")
    print(f"R2={r2:.4f}")
    print(f"Training samples={len(train_df)}")
    print(f"Validation samples={len(val_df)}")
    print(f"Features={x_train.shape[1]}")

```

Overwriting src/train.py

7. Training step

```

In [14]: from sagemaker.xgboost.estimator import XGBoost
from sagemaker.inputs import TrainingInput
from sagemaker.workflow.steps import TrainingStep

xgb_estimator = XGBoost(
    entry_point="train.py",
    source_dir="src",
    framework_version="1.7-1",
    py_version="py3",
    role=role,
    instance_type=training_instance_type,
    instance_count=1,
    hyperparameters={
        "max_depth": 6,
        "eta": 0.2,
        "subsample": 0.8,
        "objective": "reg:squarederror",
        "num_round": 400,
        "eval_metric": "rmse",
    },

    # Pass the SageMaker MLflow App information into the training container.
    # train.py uses this ARN as its MLflow tracking URI.
    environment={
        "MLFLOW_APP_ARN": MLFLOW_APP_ARN,
        "MLFLOW_EXPERIMENT_NAME": MLFLOW_EXPERIMENT_NAME,
    },

    sagemaker_session=pipeline_session,
)

# `xgboost` Library

assert isinstance(xgb_estimator, XGBoost), (
    f"xgb_estimator is a {type(xgb_estimator)}, expected "
    f"sagemaker.xgboost.estimator.XGBoost. This usually means the "
    f"kernel has stale state or a name collision with the local "
    f"xgboost library -- restart the kernel and run all cells in order."
)

```

```

)

step_train = TrainingStep(
    name="TrainXGBoostModel",
    estimator=xgb_estimator,
    inputs={
        "train": TrainingInput(
            s3_data=step_process.properties.ProcessingOutputConfig.Outputs["train"].S3Output.S3Uri,
            content_type="text/csv",
        ),
        "validation": TrainingInput(
            s3_data=step_process.properties.ProcessingOutputConfig.Outputs["validation"].S3Output.S3Uri,
            content_type="text/csv",
        ),
    },
)

```

/opt/conda/lib/python3.12/site-packages/sagemaker/estimator.py:588: SageMakerV2DeprecationWarning: XGBoost is part of the SageMaker Python SDK v2, which is on path of deprecation. In v3, use `ModelTrainer` (`from sagemaker.train import ModelTrainer`). See <https://github.com/aws/sagemaker-python-sdk/blob/master/migration.md> for the migration guide. Set SAGEMAKER_SUPPRESS_V2_WARNING=1 to silence this warning.

warn_v2_deprecation(
XGBoost is part of the SageMaker Python SDK v2, which is on path of deprecation. In v3, use `ModelTrainer` (`from sagemaker.train import ModelTrainer`). See <https://github.com/aws/sagemaker-python-sdk/blob/master/migration.md> for the migration guide. Set SAGEMAKER_SUPPRESS_V2_WARNING=1 to silence this warning.

The input argument instance_type of function (sagemaker.image_uris.retrieve) is a pipeline variable (<class 'sagemaker.workflow.parameters.ParameterString'>), which is interpreted in pipeline execution time only. As the function needs to evaluate the argument value in SDK compile time, the default_value of this Parameter object will be used to override it. Please make sure the default_value is valid.

9. Evaluation step

```

In [15]: %%writefile src/evaluation.py
import json
import os
import tarfile
import numpy as np
import pandas as pd
import xgboost as xgb
from sklearn.metrics import mean_squared_error, r2_score

model_dir = "/opt/ml/processing/model"
with tarfile.open(os.path.join(model_dir, "model.tar.gz")) as tar:
    tar.extractall(path=model_dir)

model = xgb.XGBRegressor()
model_path = os.path.join(model_dir, "xgboost-model.ubj")
if not os.path.exists(model_path):
    model_path = os.path.join(model_dir, "xgboost-model.json")
model.load_model(model_path)

test_dir = "/opt/ml/processing/test"
test_file = [f for f in os.listdir(test_dir) if f.endswith(".csv")][0]
test_df = pd.read_csv(os.path.join(test_dir, test_file), header=None)
y_test = test_df.iloc[:, 0]
x_test = test_df.iloc[:, 1:]
predictions = model.predict(x_test)

rmse = float(np.sqrt(mean_squared_error(y_test, predictions)))
r2 = float(r2_score(y_test, predictions))
report = {"regression_metrics": {"rmse": {"value": rmse, "standard_deviation": "NaN"}, "r2": {"value": r2, "standard_deviation": "NaN"}}, "test_data_split": "train_test_split"}

output_dir = "/opt/ml/processing/evaluation"
os.makedirs(output_dir, exist_ok=True)
with open(os.path.join(output_dir, "evaluation.json"), "w") as f:
    json.dump(report, f)
print(f"RMSE={rmse:.4f} R2={r2:.4f}")

```

Overwriting src/evaluation.py

```

In [16]: from sagemaker.processing import ScriptProcessor, ProcessingInput, ProcessingOutput
from sagemaker.workflow.properties import PropertyFile

```



```

from sagemaker.workflow.steps import ProcessingStep

xgb_image = sagemaker.image_uris.retrieve(
    framework="xgboost",
    region=region,
    version="1.7-1"
)

eval_processor = ScriptProcessor(
    image_uri=xgb_image, command=["python3"],
    instance_type=processing_instance_type, instance_count=1,
    role=role, sagemaker_session=pipeline_session
)

evaluation_report = PropertyFile(name="EvaluationReport", output_name="evaluation", path="evaluation.json")

step_eval = ProcessingStep(
    name="EvaluateModel",
    processor=eval_processor,
    inputs=[
        ProcessingInput(source=step_train.properties.ModelArtifacts.S3ModelArtifacts, destination="/opt/ml/processing/train"),
        ProcessingInput(source=step_process.properties.ProcessingOutputConfig.Outputs["test"].S3Output, destination="/opt/ml/processing/test"),
    ],
    outputs=[ProcessingOutput(output_name="evaluation", source="/opt/ml/processing/evaluation")],
    code="src/evaluation.py",
    property_files=[evaluation_report],
)

```

8. Adding Custom inference script to accept both JSON + CSV input with the model

In [17]: `%%writefile src/inference.py`

```

#Custom SageMaker inference script for the Singapore Rent Price XGBoost model.

import os
import json
from io import StringIO

import pandas as pd
import xgboost as xgb

# DISTRICT CATEGORIES
# Singapore postal districts present in the training data.
# (24 is absent – not a valid postal district.)

DISTRICTS = [
    1, 2, 3, 4, 5, 6, 7, 8, 9, 10,
    11, 12, 13, 14, 15, 16, 17, 18, 19,
    20, 21, 22, 23, 25, 26, 27, 28,
]

# LOGICAL FEATURE NAMES
#[target] + [bedroom, area_sm, lease_year, lease_month] + dist_1..dist_28

FEATURE_NAMES = [
    "bedroom",
    "area_sm",
    "lease_year",
    "lease_month",
] + [f"dist_{d}" for d in DISTRICTS]

EXPECTED_FEATURE_COUNT = len(FEATURE_NAMES) # 31 columns

# MODEL LOADING

```

```

def model_fn(model_dir):
    # train.py saves both text/csv and .json is the portable/preferred format
    model_path = os.path.join(model_dir, "xgboost-model.json")

    print("Loading XGBoost model")
    print("Model directory:", model_dir)
    print("Model path:", model_path)
    print("Files in model directory:", end=" ")
    try:
        print(os.listdir(model_dir))
    except Exception as e:
        print(f"(unable to list: {e})")

    if not os.path.exists(model_path):
        # Fall back to the .ubj artifact if .json isn't present for any reason.
        fallback_path = os.path.join(model_dir, "xgboost-model.ubj")
        if os.path.exists(fallback_path):
            model_path = fallback_path
        else:
            raise FileNotFoundError(
                f"Model file not found at {model_path} or {fallback_path}. "
                f"Contents of model_dir: {os.listdir(model_dir)}"
            )

    model = xgb.XGBRegressor()
    model.load_model(model_path)
    print("XGBoost model loaded successfully")

    booster = model.get_booster()
    print("Number of model features:", booster.num_features())
    print("Model feature names:", booster.feature_names)

    if booster.num_features() != EXPECTED_FEATURE_COUNT:
        raise ValueError(
            f"Expected model to have {EXPECTED_FEATURE_COUNT} features, "
            f"but model has {booster.num_features()}"
        )

    return model

# NORMALISE A JSON RECORD
# Accepts either the fully expanded one-hot form, or the compact
# form with a single "district" key that gets expanded here.

def _normalise_record(record):
    if not isinstance(record, dict):
        raise ValueError("Each JSON record must be an object.")

    record = dict(record)

    if "district" in record:
        district = int(record.pop("district"))
        if district not in DISTRICTS:
            raise ValueError(
                f"Invalid district {district}. Valid districts are {DISTRICTS}"
            )
        for d in DISTRICTS:
            record[f"dist_{d}"] = 1 if d == district else 0

    return record

# VALIDATE INPUT
# bedroom and area_sm must be whole numbers.

def _validate_whole_numbers(df):
    for col in ("bedroom", "area_sm"):
        if col in df.columns:
            values = pd.to_numeric(df[col], errors="raise")
            if (values != values.round(0)).any():
                raise ValueError(f"{col} must contain whole numbers only")

```

```

# INPUT FUNCTION
# Supports application/json and text/csv input

def input_fn(request_body, content_type):

    print("INPUT REQUEST")

    print("Content type:", content_type)

    if content_type == "application/json":
        if isinstance(request_body, bytes):
            request_body = request_body.decode("utf-8")

        data = json.loads(request_body)

        if isinstance(data, dict):
            records = [_normalise_record(data)]
        elif isinstance(data, list):
            records = [_normalise_record(r) for r in data]
        else:
            raise ValueError("JSON must be an object or a list of objects.")

        df = pd.DataFrame(records)

        # Fill any omitted feature columns (unselected districts) with 0
        for column in FEATURE_NAMES:
            if column not in df.columns:
                df[column] = 0

        # Enforce the exact column order the model was trained on
        df = df[FEATURE_NAMES]
        _validate_whole_numbers(df)

        print("JSON dataframe shape:", df.shape)
        print("Logical feature columns:", list(df.columns))
        return df

    if content_type == "text/csv":
        if isinstance(request_body, bytes):
            request_body = request_body.decode("utf-8")

        df = pd.read_csv(StringIO(request_body), header=None)

        if df.shape[1] == 5:
            # Compact format, matching the JSON input's shape:
            # bedroom, area_sm, lease_year, lease_month, district
            # district is expanded into the one-hot dist_* columns here,
            # so callers don't have to build all 31 values by hand.
            df.columns = ["bedroom", "area_sm", "lease_year", "lease_month", "district"]

            district_col = pd.to_numeric(df["district"], errors="raise").astype(int)
            invalid = ~district_col.isin(DISTRICTS)
            if invalid.any():
                raise ValueError(
                    f"Invalid district value(s): {district_col[invalid].tolist()}. "
                    f"Valid districts are {DISTRICTS}"
                )

            for d in DISTRICTS:
                df[f"dist_{d}"] = (district_col == d).astype(int)

            df = df.drop(columns=["district"])
            df = df[FEATURE_NAMES]

        elif df.shape[1] == EXPECTED_FEATURE_COUNT:
            # Fully expanded format: 4 numeric features + 27 one-hot
            df.columns = FEATURE_NAMES

        else:
            raise ValueError(
                f"CSV must contain either 5 columns (bedroom, area_sm, "
                f"lease_year, lease_month, district) or "

```

```

        f"{EXPECTED_FEATURE_COUNT} fully expanded feature columns. "
        f"Received {df.shape[1]}."
    )

    _validate_whole_numbers(df)

    print("CSV dataframe shape:", df.shape)
    print("Logical feature columns:", list(df.columns))
    return df

    raise ValueError(
        f"Unsupported content type: {content_type}. "
        f"Supported types are 'application/json' and 'text/csv'."
    )

# PREDICTION

def predict_fn(input_data, model):

    print("PREDICTION")

    input_data = input_data.copy()

    if input_data.shape[1] != EXPECTED_FEATURE_COUNT:
        raise ValueError(
            f"Expected {EXPECTED_FEATURE_COUNT} input features, "
            f"received {input_data.shape[1]}"
        )

    # column labels train.py's DataFrame had at fit time (headerless CSV,
    # columns 1..31 after dropping the target column)
    input_data.columns = [str(i) for i in range(1, EXPECTED_FEATURE_COUNT + 1)]

    print("Columns passed to XGBoost:", list(input_data.columns))
    print("Input shape:", input_data.shape)

    prediction = model.predict(input_data)
    print("Raw prediction (thousands scale):", prediction)
    return prediction

# OUTPUT FUNCTION
# Model was trained on rent scaled to thousands (mth_rent / 1000),

def output_fn(prediction, accept):

    print("OUTPUT")

    # Convert prediction back from thousands to dollars
    predictions = [round(float(p) * 1000) for p in prediction.tolist()]

    print("Prediction values ($):", predictions)

    if accept == "text/csv":
        return ",".join(str(p) for p in predictions), "text/csv"

    return json.dumps(predictions), "application/json"

```

Overwriting src/inference.py

SageMaker Model using the training artifact and `src/inference.py`, then register that model. This preserves the custom serving code in the model package.

```

In [18]: from sagemaker.xgboost.model import XGBoostModel
from sagemaker.model_metrics import ModelMetrics, MetricsSource
from sagemaker.workflow.step_collections import RegisterModel
from sagemaker.workflow.condition_step import ConditionStep
from sagemaker.workflow.conditions import ConditionGreaterThanOrEqualTo
from sagemaker.workflow.functions import JsonGet, Join

```

```

from sagemaker.workflow.fail_step import FailStep

# XGBoost serving image

xgb_image = sagemaker.image_uris.retrieve(
    framework="xgboost",
    region=region,
    version="1.7-1"
)

# Framework-specific XGBoost model

rent_model = XGBoostModel(
    image_uri=xgb_image,
    model_data=step_train.properties.ModelArtifacts.S3ModelArtifacts,
    role=role,

    # Custom inference code
    entry_point="inference.py",
    source_dir="src",

    framework_version="1.7-1",

    sagemaker_session=pipeline_session,
)

# Evaluation metrics

model_metrics = ModelMetrics(
    model_statistics=MetricsSource(
        s3_uri=Join(
            on="/",
            values=[
                step_eval.properties
                    .ProcessingOutputConfig
                    .Outputs["evaluation"]
                    .S3Output
                    .S3Uri,
                "evaluation.json",
            ],
        ),
        content_type="application/json",
    )
)

# Register model

step_register = RegisterModel(
    name="RegisterModelV2",

    model=rent_model,

    content_types=[
        "application/json",
        "text/csv",
    ],

    response_types=[
        "application/json",
        "text/csv",
    ],

    inference_instances=[
        INFERENCE_INSTANCE

```

```

    ],

    transform_instances=[
        INFERENCE_INSTANCE
    ],

    model_package_group_name=MODEL_PACKAGE_GROUP,

    approval_status=model_approval_status,

    model_metrics=model_metrics,
)

# Fail step

step_fail = FailStep(
    name="ModelQualityFailed",
    error_message="Model R2 below threshold.",
)

# R2 quality gate

step_cond = ConditionStep(
    name="CheckR2Threshold",

    conditions=[
        ConditionGreaterThanOrEqualTo(
            left=JsonGet(
                step_name=step_eval.name,
                property_file=evaluation_report,
                json_path="regression_metrics.r2.value",
            ),
            right=r2_threshold,
        )
    ],

    if_steps=[
        step_register
    ],

    else_steps=[
        step_fail
    ],
)

```

/opt/conda/lib/python3.12/site-packages/sagemaker/model.py:347: SageMakerV2DeprecationWarning: XGBoostModel is part of the SageMaker Python SDK v2, which is on path of deprecation. In v3, use `ModelBuilder` (`from sagemaker.serve import ModelBuilder`).

See <https://github.com/aws/sagemaker-python-sdk/blob/master/migration.md> for the migration guide. Set SAGEMAKER_SUPPRESS_V2_WARNING=1 to silence this warning.

```
warn_v2_deprecation(
```

XGBoostModel is part of the SageMaker Python SDK v2, which is on path of deprecation. In v3, use `ModelBuilder` (`from sagemaker.serve import ModelBuilder`).

See <https://github.com/aws/sagemaker-python-sdk/blob/master/migration.md> for the migration guide. Set SAGEMAKER_SUPPRESS_V2_WARNING=1 to silence this warning.

/opt/conda/lib/python3.12/site-packages/sagemaker/estimator.py:588: SageMakerV2DeprecationWarning: SKLearn is part of the SageMaker Python SDK v2, which is on path of deprecation. In v3, use `ModelTrainer` (`from sagemaker.train import ModelTrainer`).

See <https://github.com/aws/sagemaker-python-sdk/blob/master/migration.md> for the migration guide. Set SAGEMAKER_SUPPRESS_V2_WARNING=1 to silence this warning.

```
warn_v2_deprecation(
```

SKLearn is part of the SageMaker Python SDK v2, which is on path of deprecation. In v3, use `ModelTrainer` (`from sagemaker.train import ModelTrainer`).

See <https://github.com/aws/sagemaker-python-sdk/blob/master/migration.md> for the migration guide. Set SAGEMAKER_SUPPRESS_V2_WARNING=1 to silence this warning.

12. Build and run the pipeline

```
In [19]: from sagemaker.workflow.pipeline import Pipeline
```

```
pipeline = Pipeline(  
    name=PIPELINE_NAME,  
    parameters=[input_data, processing_instance_type, training_instance_type, model_approval_status, r2],  
    steps=[step_process, step_train, step_eval, step_cond],  
    sagemaker_session=pipeline_session,  
)  
  
pipeline.upsert(role_arn=role)  
print("Pipeline upserted:", PIPELINE_NAME)  
  
execution = pipeline.start()  
print("Execution started:", execution.arn)
```

/opt/conda/lib/python3.12/site-packages/sagemaker/workflow/pipeline.py:119: SageMakerV2DeprecationWarning: Pipeline is part of the SageMaker Python SDK v2, which is on path of deprecation. In v3, use `Pipeline` (`from sagemaker.mlops.pipeline import Pipeline`).

See <https://github.com/aws/sagemaker-python-sdk/blob/master/migration.md> for the migration guide. Set SAGEMAKER_SUPPRESS_V2_WARNING=1 to silence this warning.

WARN_v2_deprecation(
WARNING:sagemaker.deprecations:Pipeline is part of the SageMaker Python SDK v2, which is on path of deprecation. In v3, use `Pipeline` (`from sagemaker.mlops.pipeline import Pipeline`).

See <https://github.com/aws/sagemaker-python-sdk/blob/master/migration.md> for the migration guide. Set SAGEMAKER_SUPPRESS_V2_WARNING=1 to silence this warning.

INFO:sagemaker.telemetry.telemetry_logging:SageMaker Python SDK will collect telemetry to help us better understand our user's needs, diagnose issues, and deliver additional features.

To opt out of telemetry, please disable via TelemetryOptOut parameter in SDK defaults config. For more information, refer to <https://sagemaker.readthedocs.io/en/stable/overview.html#configuring-and-using-defaults-with-the-sagemaker-python-sdk>.

WARNING:sagemaker.workflow.utilities:Popping out 'ProcessingJobName' from the pipeline definition by default since it will be overridden at pipeline execution time. Please utilize the PipelineDefinitionConfig to persist this field in the pipeline definition if desired.

WARNING:sagemaker.workflow.utilities:The input argument instance_type of function (sagemaker.image_uris.get_training_image_uri) is a pipeline variable (<class 'sagemaker.workflow.parameters.ParameterString'>), which is interpreted in pipeline execution time only. As the function needs to evaluate the argument value in SDK compile time, the default_value of this Parameter object will be used to override it. Please make sure the default_value is valid.

WARNING:sagemaker.workflow.utilities:The input argument instance_type of function (sagemaker.image_uris.get_training_image_uri) is a pipeline variable (<class 'sagemaker.workflow.parameters.ParameterString'>), which is interpreted in pipeline execution time only. As the function needs to evaluate the argument value in SDK compile time, the default_value of this Parameter object will be used to override it. Please make sure the default_value is valid.

WARNING:sagemaker.workflow.utilities:Popping out 'TrainingJobName' from the pipeline definition by default since it will be overridden at pipeline execution time. Please utilize the PipelineDefinitionConfig to persist this field in the pipeline definition if desired.

WARNING:sagemaker.workflow.utilities:Popping out 'ProcessingJobName' from the pipeline definition by default since it will be overridden at pipeline execution time. Please utilize the PipelineDefinitionConfig to persist this field in the pipeline definition if desired.

WARNING:sagemaker.workflow.utilities:Popping out 'TrainingJobName' from the pipeline definition by default since it will be overridden at pipeline execution time. Please utilize the PipelineDefinitionConfig to persist this field in the pipeline definition if desired.

WARNING:sagemaker.workflow._utils:Popping out 'CertifyForMarketplace' from the pipeline definition since it will be overridden in pipeline execution time.

WARNING:sagemaker.workflow.utilities:Popping out 'ModelPackageName' from the pipeline definition by default since it will be overridden at pipeline execution time. Please utilize the PipelineDefinitionConfig to persist this field in the pipeline definition if desired.

WARNING:sagemaker.workflow.utilities:Popping out 'ProcessingJobName' from the pipeline definition by default since it will be overridden at pipeline execution time. Please utilize the PipelineDefinitionConfig to persist this field in the pipeline definition if desired.

WARNING:sagemaker.workflow.utilities:The input argument instance_type of function (sagemaker.image_uris.get_training_image_uri) is a pipeline variable (<class 'sagemaker.workflow.parameters.ParameterString'>), which is interpreted in pipeline execution time only. As the function needs to evaluate the argument value in SDK compile time, the default_value of this Parameter object will be used to override it. Please make sure the default_value is valid.

WARNING:sagemaker.workflow.utilities:Popping out 'TrainingJobName' from the pipeline definition by default since it will be overridden at pipeline execution time. Please utilize the PipelineDefinitionConfig to persist this field in the pipeline definition if desired.

WARNING:sagemaker.workflow.utilities:Popping out 'ProcessingJobName' from the pipeline definition by default since it will be overridden at pipeline execution time. Please utilize the PipelineDefinitionConfig to persist this field in the pipeline definition if desired.

WARNING:sagemaker.workflow.utilities:Popping out 'TrainingJobName' from the pipeline definition by default since it will be overridden at pipeline execution time. Please utilize the PipelineDefinitionConfig to persist this field in the pipeline definition if desired.

WARNING:sagemaker.workflow._utils:Popping out 'CertifyForMarketplace' from the pipeline definition since it will be overridden in pipeline execution time.

WARNING:sagemaker.workflow.utilities:Popping out 'ModelPackageName' from the pipeline definition by default since it will be overridden at pipeline execution time. Please utilize the PipelineDefinitionConfig to persist this field in the pipeline definition if desired.

INFO:sagemaker.telemetry.telemetry_logging:SageMaker Python SDK will collect telemetry to help us better understand our user's needs, diagnose issues, and deliver additional features.

To opt out of telemetry, please disable via TelemetryOptOut parameter in SDK defaults config. For more information, refer to <https://sagemaker.readthedocs.io/en/stable/overview.html#configuring-and-using-defaults-with-the-sagemaker-python-sdk>.

Pipeline upserted: 4697566D-Rental-Pipeline-V2

Execution started: arn:aws:sagemaker:ap-southeast-1:044528205969:pipeline/4697566D-Rental-Pipeline-V2/execution/adudsz16ady0

11A. View the MLflow App run

The training step now sends its XGBoost parameters, validation metrics, and model artifact to the SageMaker MLflow App. The code below creates a presigned MLflow App URL so you can open the experiment in the MLflow UI.

```
In [20]: # Open the SageMaker MLflow App UI
# =====
# AWS CLIENTS
# =====

sm_client = boto3.client("sagemaker", region_name=REGION)
s3_client = boto3.client("s3", region_name=REGION)
s3_resource = boto3.resource("s3", region_name=REGION)

mlflow_url_response = sm_client.create_presigned_mlflow_app_url(
    Arn=MLFLOW_APP_ARN
)

mlflow_ui_url = mlflow_url_response["AuthorizedUrl"]

print("MLflow App:", MLFLOW_APP_NAME)
print("MLflow Experiment:", MLFLOW_EXPERIMENT_NAME)
print("\nOpen this URL in your browser to view the MLflow run:")
print(mlflow_ui_url)
```



```
MLflow App: iti113-26s1-team08-mlflow-app
MLflow Experiment: ITI113/Team08/Experiment
```

Open this URL in your browser to view the MLflow run:

[illegible]

13. Deploy the latest approved model

Because the registered model now contains the custom inference specification, deployment through the Model Registry can use that specification.

```
In [22]: import boto3

sm = boto3.client("sagemaker", region_name=region)

response = sm.list_model_packages(
    ModelPackageGroupName=MODEL_PACKAGE_GROUP,
    SortBy="CreationTime",
    SortOrder="Descending",
    MaxResults=5
)

for p in response["ModelPackageSummaryList"]:
    print(
        p["ModelPackageArn"],
        p["ModelPackageStatus"],
        p.get("ModelApprovalStatus"),
        p["CreationTime"]
    )
```

```

arn:aws:sagemaker:ap-southeast-1:044528205969:model-package/RentalPriceModelGroupV2/26 Completed Approved
2026-08-26 13:59:32.167000+00:00
arn:aws:sagemaker:ap-southeast-1:044528205969:model-package/RentalPriceModelGroupV2/25 Completed Approved
2026-08-26 12:14:12.825000+00:00
arn:aws:sagemaker:ap-southeast-1:044528205969:model-package/RentalPriceModelGroupV2/24 Completed Approved
2026-08-24 11:46:28.967000+00:00
arn:aws:sagemaker:ap-southeast-1:044528205969:model-package/RentalPriceModelGroupV2/23 Completed Approved
2026-08-21 14:42:19.108000+00:00
arn:aws:sagemaker:ap-southeast-1:044528205969:model-package/RentalPriceModelGroupV2/22 Completed Approved
2026-08-21 14:20:35.688000+00:00

```

```

In [23]: from sagemaker import ModelPackage

deploy_session = sagemaker.Session()
sm_client = deploy_session.sagemaker_client

response = sm_client.list_model_packages(
    ModelPackageGroupName=MODEL_PACKAGE_GROUP,
    ModelApprovalStatus="Approved",
    SortBy="CreationTime",
    SortOrder="Descending",
    MaxResults=1,
)
packages = response["ModelPackageSummaryList"]
if not packages:
    raise RuntimeError(f"No Approved Model Package found in {MODEL_PACKAGE_GROUP}")

latest = packages[0]
model_package_arn = latest["ModelPackageArn"]
model_package_version = latest["ModelPackageVersion"]
endpoint_name = f"sgp-rental-price-endpoint-v2-{model_package_version}"

print("Model package:", model_package_arn)
print("Endpoint:", endpoint_name)

model_from_registry = ModelPackage(
    role=role,
    model_package_arn=model_package_arn,
    sagemaker_session=deploy_session,
)

predictor = model_from_registry.deploy(
    initial_instance_count=1,
    instance_type=INFERENCE_INSTANCE,
    endpoint_name=endpoint_name,
)
print("Deployment successful:", endpoint_name)

```

```

/opt/conda/lib/python3.12/site-packages/sagemaker/model.py:347: SageMakerV2DeprecationWarning: ModelPackage
is part of the SageMaker Python SDK v2, which is on path of deprecation. In v3, use `ModelBuilder` (`f
rom sagemaker.serve import ModelBuilder`).
See https://github.com/aws/sagemaker-python-sdk/blob/master/migration.md for the migration guide. Set SAG
EMAKER_SUPPRESS_V2_WARNING=1 to silence this warning.

```

```

warn_v2_deprecation(
WARNING:sagemaker.deprecations:ModelPackage is part of the SageMaker Python SDK v2, which is on path of d
eprecation. In v3, use `ModelBuilder` (`from sagemaker.serve import ModelBuilder`).
See https://github.com/aws/sagemaker-python-sdk/blob/master/migration.md for the migration guide. Set SAG
EMAKER_SUPPRESS_V2_WARNING=1 to silence this warning.

```

```

INFO:sagemaker:Creating model with name: RentalPriceModelGroupV2-2026-08-26-14-00-08-294
Model package: arn:aws:sagemaker:ap-southeast-1:044528205969:model-package/RentalPriceModelGroupV2/26
Endpoint: sgp-rental-price-endpoint-v2-26

```

```

INFO:sagemaker:Creating endpoint-config with name sgp-rental-price-endpoint-v2-26

```

```

INFO:sagemaker:Creating endpoint with name sgp-rental-price-endpoint-v2-26

```

```

-
-
-
-
-
-
!
Deployment successful: sgp-rental-price-endpoint-v2-26

```

```

In [24]: #check status of pipeline

```

```
import boto3

sm_client = boto3.client("sagemaker", region_name=region) # same `region` var your notebook already use

response = sm_client.list_pipelines(PipelineNamePrefix="4697566D")
for p in response["PipelineSummaries"]:
    print(p["PipelineName"], "|", p["PipelineArn"], "|", p.get("LastExecutionStatus", "N/A"))
```

```
4697566D-Rental-Pipeline-V2 | arn:aws:sagemaker:ap-southeast-1:044528205969:pipeline/4697566D-Rental-Pipeline-V2 | N/A
4697566D-Rental-Pipeline-V1 | arn:aws:sagemaker:ap-southeast-1:044528205969:pipeline/4697566D-Rental-Pipeline-V1 | N/A
```

Status of Containers

```
In [25]: MODEL_PACKAGE_ARN=model_package_arn
package = sm.describe_model_package(
    ModelPackageName=MODEL_PACKAGE_ARN
)

print("Status:")
print(package["ModelPackageStatus"])

print("\nApproval:")
print(package.get("ModelApprovalStatus"))

print("\nContent types:")
print(package.get("InferenceSpecification", {}).get("SupportedContentTypes"))

print("\nResponse types:")
print(package.get("InferenceSpecification", {}).get("SupportedResponseMIMETypes"))

print("\nContainers:")
for container in package.get("InferenceSpecification", {}).get("Containers", []):
    print("\nImage:")
    print(container.get("Image"))

    print("ModelDataUrl:")
    print(container.get("ModelDataUrl"))

    print("Environment:")
    print(container.get("Environment"))
```

Status:
Completed

Approval:
Approved

Content types:
['application/json', 'text/csv']

Response types:
['application/json', 'text/csv']

Containers:

```
Image:
121021644041.dkr.ecr.ap-southeast-1.amazonaws.com/sagemaker-xgboost:1.7-1
ModelDataUrl:
s3://sagemaker-ap-southeast-1-044528205969/pipelines-adusdz16ady0-xgboost-RepackModel-eNGdh3h63G/output/model.tar.gz
Environment:
{'SAGEMAKER_CONTAINER_LOG_LEVEL': '20', 'SAGEMAKER_PROGRAM': 'inference.py', 'SAGEMAKER_REGION': 'ap-southeast-1', 'SAGEMAKER_SUBMIT_DIRECTORY': 's3://sagemaker-ap-southeast-1-044528205969/sagemaker-xgboost-2026-08-26-13-41-00-833/sourcedir.tar.gz'}
```

**** Post Deployment Check, JSON inference and Test Data Input of endpoint ****

```
In [26]: import sys
import os

sys.path.insert(0, "src")

print("Python:", sys.version)
```

```
import inference

print("inference.py imported successfully")
print("Feature count:", len(inference.FEATURE_NAMES))
print("Features:", inference.FEATURE_NAMES)
```

Python: 3.12.13 | packaged by conda-forge | (main, Mar 5 2026, 16:50:00) [GCC 14.3.0]

inference.py imported successfully

Feature count: 31

Features: ['bedroom', 'area_sm', 'lease_year', 'lease_month', 'dist_1', 'dist_2', 'dist_3', 'dist_4', 'dist_5', 'dist_6', 'dist_7', 'dist_8', 'dist_9', 'dist_10', 'dist_11', 'dist_12', 'dist_13', 'dist_14', 'dist_15', 'dist_16', 'dist_17', 'dist_18', 'dist_19', 'dist_20', 'dist_21', 'dist_22', 'dist_23', 'dist_24', 'dist_25', 'dist_26', 'dist_27', 'dist_28']

```
In [27]: import json
import boto3
import pandas as pd
import numpy as np

ENDPOINT_NAME = endpoint_name # update to match your redeployed endpoint name
REGION = region # use the dynamically detected region from setup, not a hardcoded literal

DISTRICT_CATEGORIES = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16,
                        17, 18, 19, 20, 21, 22, 23, 25, 26, 27, 28]

#Payload only sends the bedroom,area_sm, lease_month, lease_year and key which only sends the selected d

def build_payload(bedroom: int, area_sm: int, lease_month: int, lease_year: int, district: int) -> dict:

    if district not in DISTRICT_CATEGORIES:
        raise ValueError(f"District {district} is not a recognized category.")
    if float(bedroom) != int(bedroom):
        raise ValueError(f"bedroom must be a whole number, got {bedroom}")
    if float(area_sm) != int(area_sm):
        raise ValueError(f"area_sm must be a whole number, got {area_sm}")
    return {
        "bedroom": int(bedroom),
        "area_sm": int(area_sm),
        "lease_month": lease_month,
        "lease_year": lease_year,
        f"dist_{district}": 1,
    }

# Return from predictor is in full integer sum e.g. $5,050 instead of 5.05 thousand

def predict(runtime, payload: dict) -> str:

    response = runtime.invoke_endpoint(
        EndpointName=ENDPOINT_NAME,
        ContentType="application/json",
        Accept="application/json",
        Body=json.dumps(payload),
    )
    result = json.loads(response["Body"].read().decode("utf-8"))
    return result[0]

#text / csv is sent instead of application/json

def predict_csv(runtime, payload: dict) -> str:

    full_payload = {
        "bedroom": payload["bedroom"],
        "area_sm": payload["area_sm"],
        "lease_year": payload["lease_year"],
        "lease_month": payload["lease_month"],
    }
    for d in DISTRICT_CATEGORIES:
        key = f"dist_{d}"
        full_payload[key] = payload.get(key, 0)

    csv_body = ",".join(str(v) for v in full_payload.values()) # values only, no header

    response = runtime.invoke_endpoint(
        EndpointName=ENDPOINT_NAME,
```

```

        ContentType="text/csv",
        Accept="application/json",
        Body=csv_body,
    )
    result = json.loads(response["Body"].read().decode("utf-8"))
    return result[0]

#Formatted output back into plain float

def parse_dollar_string(s: str) -> float:

    return float(s.replace("$", "").replace(",", ""))

def run_manual_test_cases():
    runtime = boto3.client("sagemaker-runtime", region_name=REGION)
    """
    test_cases = [
        {"bedroom": 3, "area_sm": 90, "lease_month": 6, "lease_year": 2024, "district": 5},
        {"bedroom": 5, "area_sm": 200, "lease_month": 1, "lease_year": 2024, "district": 10},
        {"bedroom": 1, "area_sm": 40, "lease_month": 12, "lease_year": 2023, "district": 15},
    ]
    """
    test_cases = [
        {"bedroom": 3, "area_sm": 90, "lease_year": 2024, "lease_month": 6, "district": 5},
        {"bedroom": 5, "area_sm": 200, "lease_year": 2024, "lease_month": 1, "district": 10},
        {"bedroom": 1, "area_sm": 40, "lease_year": 2023, "lease_month": 12, "district": 15},
    ]

    for i, case in enumerate(test_cases, 1):
        payload = build_payload(**case)
        predicted_rent = predict(runtime, payload) # already formatted
        print(f"Test case {i}: {case}")
        print(f"    Predicted monthly rent: {predicted_rent}\n")

#Sample real rows and compare model prediction vs actual recorded rent.

def run_real_sample_test(csv_path: str = "rental_2125.csv", n: int = 5, seed: int = 42):

    df = pd.read_csv(csv_path)
    df = df.dropna(subset=["bedroom", "area_sm", "lease date", "district", "mth rent"])
    df = df[df["district"].isin(DISTRICT_CATEGORIES)]
    df["lease_year"] = pd.to_datetime(df["lease date"]).dt.year
    df["lease_month"] = pd.to_datetime(df["lease date"]).dt.month

    sample = df.sample(n=min(n, len(df)), random_state=seed)
    runtime = boto3.client("sagemaker-runtime", region_name=REGION)

    print(f"\n{' '*60}\nComparing predictions against real recorded rents\n{' '*60}")
    for _, row in sample.iterrows():
        payload = build_payload(
            bedroom=int(row["bedroom"]),
            area_sm=int(round(row["area_sm"])),
            lease_year=int(row["lease_year"]),
            lease_month=int(row["lease_month"]),
            district=int(row["district"]),
        )
        predicted_rent_str = predict(runtime, payload) # already formatted
        predicted_monthly_rent = round(parse_dollar_string(predicted_rent_str))
        actual_monthly_rent = round(float(row["mth rent"]) * 1000)

        print(f"District {int(row['district'])}, {int(row['bedroom'])}BR, {row['area_sm']}sqm")
        print(f"    Predicted:{predicted_rent_str} | Actual: ${actual_monthly_rent:,.0f} "
              f"| Diff: ${predicted_monthly_rent - actual_monthly_rent:,.0f}")

if __name__ == "__main__":
    print("Running manual test cases (application/json)...\n")
    run_manual_test_cases()

    print("\nSame prediction via text/csv, for comparison...")

```



```
def invoke_json():
    payload = {"bedroom": 3, "area_sm": 90, "lease_year": 2024, "lease_month": 6, "district": 5}
    response = runtime.invoke_endpoint(EndpointName=endpoint_name, ContentType="application/json", Accept="application/json")
    return response["Body"].read().decode()

print("CSV result :", invoke_csv())
print("JSON result:", invoke_json())
```

CSV result : [4995]

JSON result: [4995]

Model success criteria

- Processing completes.
- Training completes and produces `xgboost-model.ubj`.
- Evaluation produces `evaluation.json`.
- R^2 threshold condition passes.
- Model Registry contains a V2 model package.
- The model package includes the custom `inference.py` serving specification.
- Endpoint accepts `text/csv`.
- Endpoint accepts `application/json`.
- Compact JSON with `district` is converted to the one-hot feature vector expected by the model.

Pause for Endpoint Testing before removal

Testing the endpoint from browser

In [30]: `!pip install streamlit`

```
Requirement already satisfied: streamlit in /opt/conda/lib/python3.12/site-packages (1.62.0)
Requirement already satisfied: altair!=5.4.0,!=5.4.1,<7,>=5.0.0 in /opt/conda/lib/python3.12/site-packages (from streamlit) (6.2.2)
Requirement already satisfied: blinker<2,>=1.5.0 in /opt/conda/lib/python3.12/site-packages (from streamlit) (1.9.0)
Requirement already satisfied: click<9,>=7.0 in /opt/conda/lib/python3.12/site-packages (from streamlit) (8.2.1)
Requirement already satisfied: numpy<3,>=1.23 in /opt/conda/lib/python3.12/site-packages (from streamlit) (1.26.4)
Requirement already satisfied: packaging>=20 in /opt/conda/lib/python3.12/site-packages (from streamlit) (24.2)
Requirement already satisfied: pandas<4,>=1.4.0 in /opt/conda/lib/python3.12/site-packages (from streamlit) (2.3.3)
Requirement already satisfied: pillow<13,>=7.1.0 in /opt/conda/lib/python3.12/site-packages (from streamlit) (11.3.0)
Requirement already satisfied: pydeck<1,>=0.8.0b4 in /opt/conda/lib/python3.12/site-packages (from streamlit) (0.9.3)
Requirement already satisfied: protobuf<8,>=5.26.1 in /opt/conda/lib/python3.12/site-packages (from streamlit) (6.31.1)
Requirement already satisfied: pyarrow!=25.0.0,<26,>=7.0 in /opt/conda/lib/python3.12/site-packages (from streamlit) (21.0.0)
Requirement already satisfied: requests<3,>=2.27 in /opt/conda/lib/python3.12/site-packages (from streamlit) (2.34.2)
Requirement already satisfied: toml<2,>=0.10.1 in /opt/conda/lib/python3.12/site-packages (from streamlit) (0.10.2)
Requirement already satisfied: typing-extensions<5,>=4.10.0 in /opt/conda/lib/python3.12/site-packages (from streamlit) (4.16.0)
Requirement already satisfied: starlette<2,>=0.46.0 in /opt/conda/lib/python3.12/site-packages (from streamlit) (0.52.1)
Requirement already satisfied: uvicorn<1,>=0.30.0 in /opt/conda/lib/python3.12/site-packages (from streamlit) (0.51.0)
Requirement already satisfied: httptools<1,>=0.6.3 in /opt/conda/lib/python3.12/site-packages (from streamlit) (0.8.0)
Requirement already satisfied: anyio<5,>=4.0.0 in /opt/conda/lib/python3.12/site-packages (from streamlit) (4.14.2)
Requirement already satisfied: python-multipart<1,>=0.0.10 in /opt/conda/lib/python3.12/site-packages (from streamlit) (0.0.32)
Requirement already satisfied: websockets<17,>=12.0.0 in /opt/conda/lib/python3.12/site-packages (from streamlit) (16.1.1)
```


Requirement already satisfied: itsdangerous<3,>=2.1.2 in /opt/conda/lib/python3.12/site-packages (from streamlit) (2.2.0)

Requirement already satisfied: watchdog<7,>=2.1.5 in /opt/conda/lib/python3.12/site-packages (from streamlit) (6.0.0)

Requirement already satisfied: jinja2 in /opt/conda/lib/python3.12/site-packages (from altair!=5.4.0,!>5.4.1,<7,>=5.0.0->streamlit) (3.1.6)

Requirement already satisfied: jsonschema>=3.0 in /opt/conda/lib/python3.12/site-packages (from altair!=5.4.0,!>5.4.1,<7,>=5.0.0->streamlit) (4.23.0)

Requirement already satisfied: narwhals>=2.4.0 in /opt/conda/lib/python3.12/site-packages (from altair!=5.4.0,!>5.4.1,<7,>=5.0.0->streamlit) (2.24.0)

Requirement already satisfied: idna>=2.8 in /opt/conda/lib/python3.12/site-packages (from anyio<5,>=4.0.0->streamlit) (3.18)

Requirement already satisfied: python-dateutil>=2.8.2 in /opt/conda/lib/python3.12/site-packages (from pandas<4,>=1.4.0->streamlit) (2.9.0.post0)

Requirement already satisfied: pytz>=2020.1 in /opt/conda/lib/python3.12/site-packages (from pandas<4,>=1.4.0->streamlit) (2024.2)

Requirement already satisfied: tzdata>=2022.7 in /opt/conda/lib/python3.12/site-packages (from pandas<4,>=1.4.0->streamlit) (2026.3)

Requirement already satisfied: charset_normalizer<4,>=2 in /opt/conda/lib/python3.12/site-packages (from requests<3,>=2.27->streamlit) (3.4.9)

Requirement already satisfied: urllib3<3,>=1.26 in /opt/conda/lib/python3.12/site-packages (from requests<3,>=2.27->streamlit) (1.26.20)

Requirement already satisfied: certifi>=2023.5.7 in /opt/conda/lib/python3.12/site-packages (from requests<3,>=2.27->streamlit) (2026.6.17)

Requirement already satisfied: h11>=0.8 in /opt/conda/lib/python3.12/site-packages (from uvicorn<1,>=0.30.0->streamlit) (0.16.0)

Requirement already satisfied: MarkupSafe>=2.0 in /opt/conda/lib/python3.12/site-packages (from jinja2->altair!=5.4.0,!>5.4.1,<7,>=5.0.0->streamlit) (3.0.3)

Requirement already satisfied: attrs>=22.2.0 in /opt/conda/lib/python3.12/site-packages (from jsonschema>=3.0->altair!=5.4.0,!>5.4.1,<7,>=5.0.0->streamlit) (25.4.0)

Requirement already satisfied: jsonschema-specifications>=2023.03.6 in /opt/conda/lib/python3.12/site-packages (from jsonschema>=3.0->altair!=5.4.0,!>5.4.1,<7,>=5.0.0->streamlit) (2025.9.1)

Requirement already satisfied: referencing>=0.28.4 in /opt/conda/lib/python3.12/site-packages (from jsonschema>=3.0->altair!=5.4.0,!>5.4.1,<7,>=5.0.0->streamlit) (0.37.0)

Requirement already satisfied: rpd-py>=0.7.1 in /opt/conda/lib/python3.12/site-packages (from jsonschema>=3.0->altair!=5.4.0,!>5.4.1,<7,>=5.0.0->streamlit) (0.27.1)

Requirement already satisfied: six>=1.5 in /opt/conda/lib/python3.12/site-packages (from python-dateutil>=2.8.2->pandas<4,>=1.4.0->streamlit) (1.17.0)

In [31]: `!pip install pyngrok`

Requirement already satisfied: pyngrok in /opt/conda/lib/python3.12/site-packages (8.1.2)

Requirement already satisfied: PyYAML>=5.1 in /opt/conda/lib/python3.12/site-packages (from pyngrok) (6.0.3)

In [32]: `from pyngrok import ngrok`

`ngrok.set_auth_token("3I4yHHTo3NjAEeAFjTDHKqhSKfR_2HJX8mLz24XirqzEgo6q")`

INFO:pyngrok.process:Updating authtoken for default "config_path" of "ngrok_path": /home/sagemaker-user/.config/ngrok/ngrok

In [34]: `import time`

`print("Application will pause for 3 mins before tearing down endpoint")`
`time.sleep(300)`
`print("Program resumed to clean up endpoint")`

Application will pause for 3 mins before tearing down endpoint
Program resumed to clean up endpoint

Clean Up*

In [35]: `# Clean up: delete the endpoint then VERIFY deletion`

`endpoint_desc = sm_client.describe_endpoint(EndpointName=endpoint_name)`
`endpoint_config_name = endpoint_desc["EndpointConfigName"]`

`sm_client.delete_endpoint(EndpointName=endpoint_name)`
`print(f"Delete requested for endpoint: {endpoint_name}")`

`sm_client.delete_endpoint_config(EndpointConfigName=endpoint_config_name)`
`print(f"Delete requested for endpoint config: {endpoint_config_name}")`

`# Verify the endpoint is actually gone – describe_endpoint should now raise a ClientError`
`from botocore.exceptions import ClientError`


```

try:
    sm_client.describe_endpoint(EndpointName=endpoint_name)
    print("WARNING: endpoint still exists – deletion may not have completed yet. Wait a few seconds and
except ClientError as e:
    if "Could not find endpoint" in str(e) or "ValidationException" in str(e):
        print(f"Verified: endpoint '{endpoint_name}' no longer exists.")
    else:
        raise

try:
    sm_client.describe_endpoint_config(EndpointConfigName=endpoint_config_name)
    print("WARNING: endpoint config still exists.")
except ClientError as e:
    if "Could not find endpoint configuration" in str(e) or "ValidationException" in str(e):
        print(f"Verified: endpoint config '{endpoint_config_name}' no longer exists.")
    else:
        raise

```

Delete requested for endpoint: sgp-rental-price-endpoint-v2-26

Delete requested for endpoint config: sgp-rental-price-endpoint-v2-26

WARNING: endpoint still exists – deletion may not have completed yet. Wait a few seconds and re-check.

Verified: endpoint config 'sgp-rental-price-endpoint-v2-26' no longer exists.

In []: